

Esercizio 4

Il termine binding si riferisce in Java a un processo di associazione di metodi per richiamare l'implementazione specifica più adeguata. Non a caso, il termine binding può essere tradotto proprio come “collegamento” o “legame”.

1. Il Binding Statico è un processo che consiste nell'associare un metodo con il tipo della classe di una variabile che contiene il riferimento a un oggetto. Questa associazione viene risolta in fase di compilazione del programma dal compilatore, che decide il metodo specifico da chiamare in base al tipo usato nella dichiarazione della variabile, non tenendo però presente quale sarà il tipo effettivo utilizzato a tempo di esecuzione. Come vantaggi offre delle prestazioni migliori, perché per esempio non deve essere effettuata alcuna risoluzione dinamica a runtime, in quanto già decisa in precedenza, e controllo di tipi. Riguardo quest'ultimo vantaggio, permette il rilevamento di errori a tempo di compilazione e anche maggiori possibilità di ottimizzazione del codice (conoscendo già i tipi delle variabili il compilatore può applicare diverse forme di ottimizzazione). Tuttavia, il Binding Statico non è sempre possibile, in quanto non permetterebbe, in determinate circostanze, di chiamare il metodo specifico appropriato o di scrivere codice generalizzato.
2. La differenza sostanziale del Binding Dinamico da quello statico è rappresentata dal momento in cui avviene la risoluzione delle chiamate ai metodi (o degli accessi alle variabili), in quanto nel caso del Binding Dinamico essa avviene a tempo di esecuzione. Questo tipo di binding è implementato dal polimorfismo, in modo da poter

chiamare un metodo senza essere a conoscenza del tipo esatto dell'oggetto su cui si sta invocando. Invece che chiamare il metodo rispetto al tipo del riferimento, la JVM lo chiamerà considerando il tipo usato per costruire l'oggetto, se implementato, altrimenti si risalirà verso la superclasse che lo implementa (o meglio, verrà usato il metodo che è stato ereditato già implementato). Il Binding Dinamico offre quindi una maggiore flessibilità, risolvendo il metodo chiamato in base al tipo reale dell'oggetto durante l'esecuzione. Con il polimorfismo, quindi, un oggetto di una sottoclasse può essere assegnato a una variabile della superclasse e quando si chiama un metodo ci si riferirà a quello definito nella sottoclasse, qualora questo sia lì implementato.

3. Il Binding Statico viene in generale applicato ai tipi primitivi. L'oggetto effettivo non viene utilizzato per questo tipo di binding. Inoltre, possiamo considerare Binding Statico quello che avviene con i metodi che sono static, private o final, in quanto non è possibile sovrascriverli (facendo Override) e quindi vengono associati a tempo di compilazione. Infatti:

- non è possibile eseguire l'override di un metodo statico in quanto per definizione questo tipo di metodi è legato alla classe, non all'istanza specifica di un oggetto;
- non è possibile per i metodi final, in quanto questo modificatore li rende immutabili;
- non è possibile per i metodi private, perché non visibili all'esterno della classe in cui sono dichiarati.

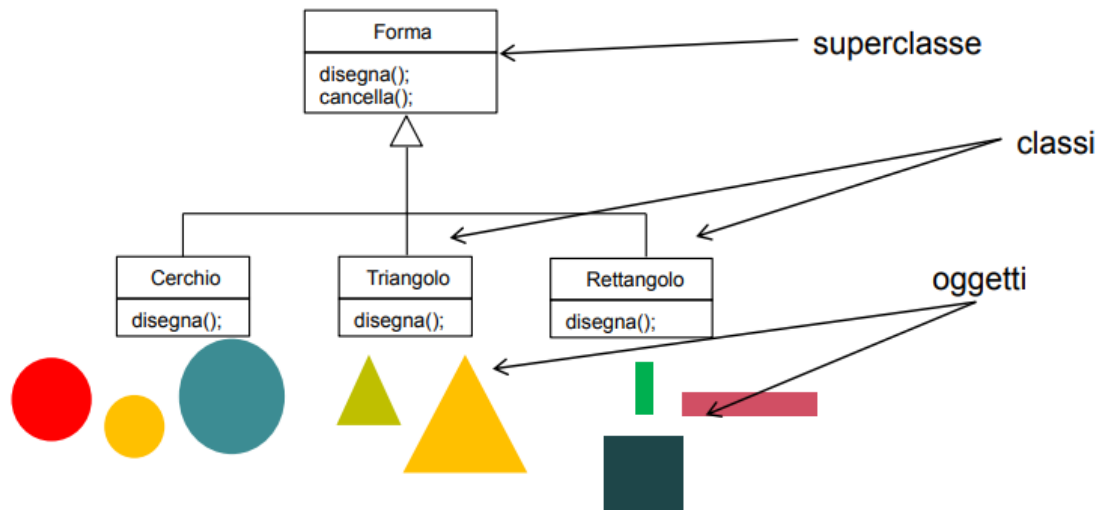
Al contrario, è possibile l'overloading dei metodi e il Binding Statico viene applicato grazie al fatto che ho più metodi con la stessa segnatura, ma che differiscono per tipo, ordine o numero di

parametri. Questo fa sì che venga scelta la versione del metodo appropriata in base alla tipologia di parametri con cui è chiamato.

4. In Java, il Binding Dinamico riguarda le chiamate ai metodi e si applica ai riferimenti degli oggetti. Viene utilizzato in tutti quei casi in cui non si ha da subito a disposizione l'informazione riguardo il tipo effettivo di un oggetto, quindi in particolare quando si lavora con l'ereditarietà e con l'overriding, in quanto ogni sottoclasse appartenente a una gerarchia di classi può fornire una propria implementazione di uno stesso metodo ereditato da una superclasse ed è importante sapere quale chiamare, ma questo si può sapere solo se si è a conoscenza del tipo specifico di un oggetto appartenente alla gerarchia al momento in cui viene effettuata la chiamata a tempo di esecuzione. La stessa cosa vale quando si hanno interfacce e si vogliono fornire implementazioni personalizzate dei metodi specifici nelle classi che le implementano. Il Binding Dinamico è quindi essenziale per il polimorfismo, in quanto permette di trattare oggetti di classi diverse come istanze di una stessa classe genitore. Permette flessibilità del codice.

Esempio:

(L'immagine della gerarchia delle classi utilizzata per gli esempi è stata ottenuta dalla slide 6 – Polimorfismo_Liste del corso di MDP 22-23, a cura del Prof. Faralli e del Prof. Navigli)



Quando si utilizza il polimorfismo, il tipo dichiarato della variabile può essere la classe genitore, ma il tipo effettivo dell'oggetto a cui fa riferimento può essere una delle sue sottoclassi. Durante l'esecuzione, il Binding Dinamico assicura che il metodo corretto della sottoclasse venga invocato.

```
class Forma {  
    public void disegna() {  
        System.out.println("Disegno una forma generica.");  
    }  
}
```

```
class Cerchio extends Forma {  
    @Override  
    public void disegna() {  
        System.out.println("Disegno un cerchio.");  
    }  
}
```

```
}  
}
```

```
class Rettangolo extends Forma {  
    @Override  
    public void disegna() {  
        System.out.println("Disegno un rettangolo.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Forma forma1 = new Cerchio();  
        Forma forma2 = new Rettangolo();  
        Forma forma3 = new Forma();  
  
        forma1.disegna(); // Stampa: "Disegno un cerchio."  
        forma2.disegna(); // Stampa: "Disegno un rettangolo."  
        forma3.disegna(); // Stampa: "Disegno una forma generica."  
  
    }  
}
```

Esempio per l'overloading:

```
class Forma {  
    public void disegna() {  
        System.out.println("Disegno una forma generica.");  
    }  
  
    public void disegna(String colore) {  
        System.out.println("Disegno una forma generica di colore " +  
colore);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Forma forma = new Forma();  
        forma.disegna();           // Stampa: "Disegno una forma generica."  
        forma.disegna("rosso");    // Stampa: "Disegno una forma  
generica di colore rosso"  
    }  
}
```

Quando chiamo `forma.disegna("rosso")`, considerando il parametro passato, il compilatore può stabilire subito quale metodo chiamare tra i due.